



Sobrio — Extension navigateur

Comment l'extension trie vos prompts et recommande un modèle Claude, et comment le code fonctionne aujourd'hui.

Version 1.3.0 · openapi v1.1 (RFC-0003) · document technique généré le 17 juillet 2026

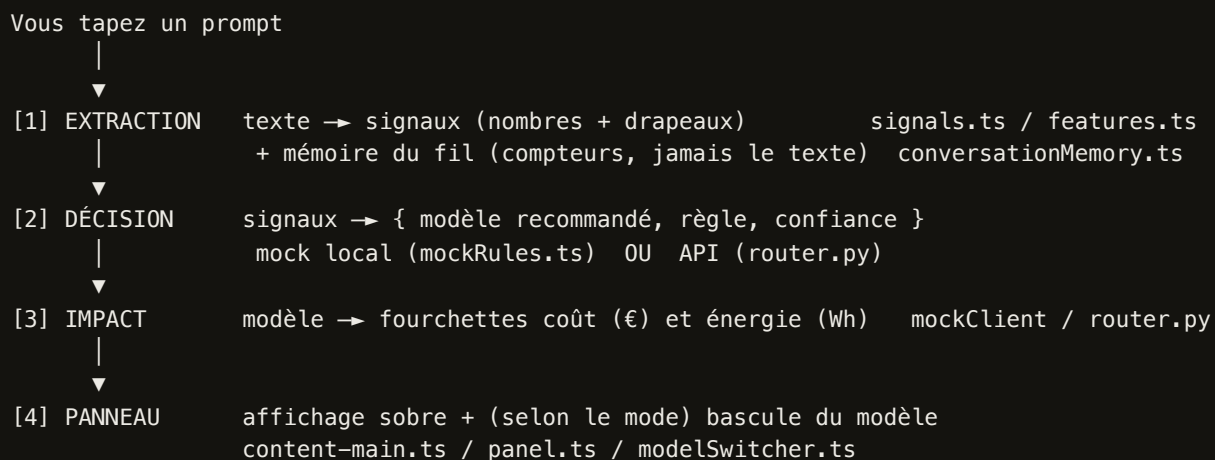
Périmètre : `extension/` (Lot A) + routeur API stub (`api/` , Lot B).

Sommaire

1. Vue d'ensemble : de la frappe à la recommandation
2. Ce qui sort de votre poste (et ce qui n'en sort jamais)
3. Étape 1 — Réduire le prompt en *signaux* (mesures & drapeaux)
4. Étape 2 — La mémoire de conversation
5. Étape 3 — Le tri : les règles de décision
6. Mock vs API réelle : deux « cerveaux »
7. Confiance, coût, énergie
8. Étape 4 — L'affichage et la bascule de modèle (`assist_mode`)
9. Exemples de bout en bout
10. Carte des fichiers

1. Vue d'ensemble : de la frappe à la recommandation

Sobrio observe la barre de saisie de `claude.ai`. Quand vous marquez une courte pause de frappe, l'extension suit un pipeline en quatre temps, entièrement local pour la partie sensible :



Le tri des prompts — le cœur de votre question — se joue aux étapes **[1]** et **[2]** : d'abord réduire le texte à des *signaux* mesurables, ensuite appliquer des *règles déterministes* à ces signaux. Il n'y a **aucun modèle de machine learning** dans la version actuelle : ce sont des heuristiques lisibles et testées, choisies pour être explicables (« Pourquoi ? » dans le panneau) et prudentes.

2. Ce qui sort de votre poste (et ce qui n'en sort jamais)

Règle n°1, non négociable : **aucun texte de prompt ou de conversation ne quitte le poste**. Le module d'extraction reçoit le texte, le réduit immédiatement en mesures/drapeaux, puis l'oublie. Les seules chaînes qui peuvent sortir appartiennent à des **vocabulaires fermés** : la langue (`fr/en/other`), les drapeaux (liste figée), les identifiants de modèle du catalogue (`claude-...`). Un test automatique « zéro texte » le vérifie (aucune chaîne > 24 caractères dans les signaux).

Concrètement : « Analyse ce *contrat confidentiel* de 4,2 M€... » ne transmet que `{ char_len: 41, token_est: 11, lang: "fr", has_code: false, has_math: false, keyword_flags: ["contrat", "analyse"] }`. Le texte lui-même reste dans votre navigateur.

3. Étape 1 — Réduire le prompt en *signaux*

Le texte saisi est transformé en un objet `PromptSignals` . Chaque champ est calculé par une fonction pure (sans effet de bord). Voici **exactement** comment :

Signal	Comment il est calculé (code réel)
<code>char_len</code>	Longueur du texte en caractères (<code>text.length</code>).
<code>token_est</code>	Estimation grossière du nombre de tokens : <code>ceil(char_len / 4)</code> . (Approximation volontaire en v0 ; un vrai tokenizer est prévu.)
<code>lang</code>	Comptage de mots vides fréquents FR vs EN (listes figées : <code>le, la, de, et...</code> / <code>the, of, and, to...</code>). Le camp majoritaire gagne ; égalité ou aucun indice $\Rightarrow$ <code>other</code> .
<code>has_code</code>	Vrai si le texte contient un motif de code (expressions régulières) : <code>```</code> (bloc), <code>`inline`</code> , <code>function/const/let/var X, def X(, class X, import/from ...</code> , <code>console.log()/print()</code> , ou <code>=></code> .
<code>has_math</code>	Vrai si un motif mathématique OU un mot du lexique maths est présent. Motifs : symboles $\sum \int \sqrt{\pi} \leq \geq \neq \pm \times \div \partial \infty$, exposants ^{2 3} , LaTeX (<code>\frac \int \sum \sqrt \lim...</code>), <code>\$\$\$</code> , opérations <code>« 3 + 4 »</code> , fractions <code>« a/b »</code> . Lexique (racines, sans accents): <code>demontr, theoreme, preuve, lemme, integrale, derivee, equation, matrice, probabilit, proof, theorem, integral, derivative, matrix</code> .
<code>keyword_flags</code>	Sous-ensemble d'une liste fermée : <code>contrat, analyse, code, resume, traduction, demonstration</code> . Détecté par racine, insensible à la casse et aux accents (<code>« Résumé » $\Rightarrow$ resume , « démontrer » $\Rightarrow$ demonstration</code>).

Normalisation commune : minuscules + suppression des accents (`é→e, ç→c`) avant toute comparaison de mots. Côté API (contrat v1.0), un champ analogue `has_attachment_hint` détecte la mention d'une pièce jointe (`« ci-joint »` , `« attached »...`).

4. Étape 2 — La mémoire de conversation

C'est la pièce qui évite le tri *naïf*. Un prompt court comme `« démontre-le »` ne doit pas partir sur le modèle le plus léger s'il arrive dans un fil où l'on faisait des maths. À chaque mise à jour, l'extension relit les bulles visibles du fil **localement**, les réduit en compteurs, puis oublie le texte. Elle vit en mémoire de session (rien n'est stocké) et se réinitialise à chaque changement de conversation.

Signal de conversation	Signification
msg_count	Nombre de bulles (messages) du fil.
context_token_est	Somme des token_est de toutes les bulles (taille du contexte).
seen_code	Vrai si une bulle du fil contenait du code.
seen_math	Vrai si une bulle contenait un signal mathématique.
seen_reasoning	Vrai si : présence de maths, OU réponse de l'assistant longue (> 400 tokens estimés), OU marqueurs de raisonnement (donc, par conséquent, étape 1/2, raisonnement, therefore, step 1/2).
current_model	Modèle actuellement sélectionné dans claude.ai, ramené à un id du catalogue par famille (« Claude Opus 4.8 » ⇒ c laude-opus-4-8); null si inconnu.
recos_shown / recos_followed / derogations_up	Compteurs d'usage du fil. derogations_up ne compte que les dérogations vers un modèle plus cher que la reco (signal d'inconfort). Ces compteurs survivent aux mises à jour d'un même fil.

Détection d'un nouveau fil (⇒ remise à zéro) : l'identifiant d'URL change, ou le nombre de bulles *régresse* quand le fil est anonyme (fil vidé).

5. Étape 3 — Le tri : les règles de décision

En mode **mock** (le défaut, sans serveur), la décision applique **quatre règles ordonnées** à l'ensemble des signaux (prompt + conversation), plus un cas par défaut. La **première règle qui correspond gagne** — l'ordre compte. Chaque décision renvoie un modèle, une *règle* (pour l'explication) et une *confiance*.

#	Règle	Condition (exacte)	Modèle	Confiance
1	code_context	prompt.has_code OU conversation.seen_code OU le drapeau code est présent.	Claude Sonnet 5	0,70
2	reasoning_context	prompt.has_math OU seen_math OU seen_reasoning OU le drapeau demonstration . (C'est la règle qui « rattrape » un prompt court dans un fil de raisonnement.)	Claude Sonnet 5	0,60 si token_est < 50 , sinon 0,75
3	short_simple	token_est < 300 ET aucun drapeau lourd (contrat / analyse) ET msg_count ≤ 6 ET context_token_est < 2000 .	Claude Haiku 4.5	0,80
4	complex_task	drapeau contrat / analyse OU token_est > 800 OU context_token_est > 4000 .	Claude Opus 4.8	0,65
—	default_balanced	Aucune des règles ci-dessus (signal peu marqué).	Claude Sonnet 5	0,55

Lecture de la logique. On protège d'abord les cas « exigeants » (code, puis raisonnement) — même sur un prompt court — car s'y tromper coûte cher en qualité. On n'aiguille vers le modèle léger (Haiku) que si *tout* indique une demande courte, simple et un fil léger. Les tâches manifestement lourdes (contrat/analyse, prompt long, gros contexte) vont au modèle le plus capable (Opus). En cas de doute, on reste au milieu (Sonnet) avec une confiance basse — et on le dit.

Suggestion « nouvelle conversation »

Indépendamment du modèle, si `context_token_est > 8000` , la réponse porte `suggest_new_conversation = true` : le panneau affiche un bandeau discret invitant à repartir d'un fil neuf (souvent moins coûteux).

6. Mock vs API réelle : deux « cerveaux »

Deux implémentations de décision coexistent, choisies par le réglage `backend` du popup :

Mode	Décideur	Utilise la mémoire de conversation ?
mock (défaut)	mockRules.ts — les 4 règles ci-dessus, entièrement local, aucun serveur.	Oui — prompt + fil (le tri le plus « intelligent »).
api	api/app/router.py (HeuristicRouterV0) — 3 règles sur les features du contrat v1.0.	Pas encore — seul le prompt courant (les signaux de conversation sont proposés dans la RFC-0001, v1.1).

Les 3 règles du routeur API (stub) :

(a) token_est < 300 ET pas de code ET pas de flag lourd	→	Haiku 4.5	(conf. 0,80)
(b) présence de code (has_code ou flag "code")	→	Sonnet 5	(conf. 0,70)
(c) sinon (long, contrat/analyse...)	→	Opus 4.8	(conf. 0,60)

À retenir : **en mode mock (le défaut aujourd'hui), c'est la mémoire du fil qui rend le tri fin** (« démontre-le » ne part pas sur Haiku). En mode API, le stub actuel ne regarde que le prompt courant — la prise en compte du fil côté serveur est formalisée dans la RFC-0001 et arrivera avec le routeur v0 complet (TODO Lot B).

7. Confiance, coût, énergie

La confiance

C'est une valeur entre 0 et 1 attachée à chaque règle (voir tableau §5). Elle sert deux buts : afficher une jauge honnête dans le panneau, et — nouveauté v1.3.0 — **décider si la bascule automatique se déclenche** : seuil par défaut **0,75**. Conséquence directe du tableau des règles :

Règle	Confiance	Bascule <code>auto</code> ? (seuil 0,75)
<code>short_simple</code> → Haiku	0,80	Oui
<code>reasoning_context</code> (prompt ≥ 50 tok) → Sonnet	0,75	Oui (au seuil)
<code>code_context</code> → Sonnet	0,70	Non → bouton « Utiliser »
<code>complex_task</code> → Opus	0,65	Non
<code>reasoning_context</code> (prompt court) → Sonnet	0,60	Non
<code>default_balanced</code> → Sonnet	0,55	Non

Coût et énergie — toujours en fourchette

Aucun chiffre d'impact n'est jamais une valeur unique (règle n°3) : toujours un *min–max* avec périmètre (« inférence »). Le coût central d'un appel se calcule à partir des prix catalogue (USD par million de tokens) :

Modèle	Entrée \$/Mtok	Sortie \$/Mtok
Claude Haiku 4.5	1	5
Claude Sonnet 5	3	15
Claude Opus 4.8	5	25
Claude Fable 5 (chiffage seul, non proposé)	10	50

Tokens de sortie estimés : `token_est` borné entre 150 et 2000. Conversion `EUR_PER_USD = 0,92` . Bande d'incertitude **±20 %** autour de la valeur centrale (min/max). L'énergie provient du module `sobrio_impact` (Range min–max obligatoire). Le prix affiché du Sonnet 5 est le tarif *durable* (3/15), pas le prix d'introduction promotionnel — un outil de maîtrise de coût doit refléter le régime permanent.

8. Étape 4 — Affichage & bascule de modèle (`assist_mode`)

Une fois la décision prise, le panneau s'affiche. La façon dont l'extension *applique* (ou non) le modèle dépend du **mode d'assistance effectif**, qui est l'intersection de votre consentement local (case du popup) et de la politique de l'organisation :

Mode	Comportement
auto	Si la confiance $\geq$ seuil (0,75) et que le modèle courant diffère de la reco : le panneau s'ouvre déjà en « Basculé sur {modèle} » (perçu < 300 ms) avec un bouton Annuler ; le modèle change réellement dans le sélecteur de claude.ai. Annuler restaure le modèle précédent.
one_click	La bascule n'a lieu qu'au clic sur « Utiliser {modèle} ». (Défaut prudent du contrat.)
guide	Aucun contact avec la page : le bouton « J'utiliserai {modèle} » note l'intention mais ne touche pas au sélecteur. C'est le repli de sûreté (kill-switch de prudence, ou case du popup décochée, ou sélecteurs cassés).

La bascule est **encadrée** et ne touche *que* le sélecteur de modèle : un seul clic simulé pour ouvrir le menu, un clic sur l'item, vérification que l'étiquette relue correspond — au moindre doute, abandon silencieux (jamais bloquant). Plusieurs garde-fous issus du durcissement de la v1.3.0 :

- **Acceptation différée** : en `auto`, le suivi n'est enregistré que lorsque vous *acceptez* (vous écartez le panneau sans annuler) — exactement un événement de télémétrie net par recommandation, jamais deux.
- **Garde de conversation** : si vous changez de fil pendant une bascule, elle s'abandonne (ni panneau fantôme, ni bascule sur le mauvais fil).
- **Sérialisation** : deux bascules rapprochées ne se disputent jamais le menu (une seule à la fois, la dernière demandée gagne).
- **Repli silencieux** vers `guide` si les sélecteurs de claude.ai changent (un signal de santé `selector_broken` est émis pour l'ops).

9. Exemples de bout en bout

Situation	Signaux déterminants	Règle → modèle	En mode auto
« Quelle heure est-il à Tokyo ? » (fil léger)	<code>token_est ≈ 8</code> , pas de <code>code/maths</code> , <code>msg_count</code> faible	<code>short_simple</code> → Haiku 4.5	Confiance 0,80 → bascule vers Haiku + Annuler
« démontre-le » dans un fil où l'on faisait des maths	prompt court, mais <code>seen_math = true</code> (mémoire du fil)	<code>reasoning_context</code> → Sonnet 5	Confiance 0,60 (prompt court) → pas de bascule auto, bouton « Utiliser »
Un message contenant un bloc <code>```python...```</code>	<code>has_code = true</code>	<code>code_context</code> → Sonnet 5	Confiance 0,70 < 0,75 → bouton « Utiliser »
« Analyse ce contrat de 40 pages ci-joint »	drapeaux <code>analyse</code> + <code>contrat</code> (lourds)	<code>complex_task</code> → Opus 4.8	Confiance 0,65 → bouton « Utiliser »
Prompt neutre, fil déjà volumineux	aucune règle franche ; <code>context_token_est</code> élevé	<code>default_balanced</code> → Sonnet 5 (+ bandeau si > 8000)	Confiance 0,55 → bouton « Utiliser »

Le deuxième exemple est le *scénario-clé* du produit : sans mémoire de conversation, un « démontre-le » de 11 caractères partirait naïvement sur Haiku. Grâce à `seen_math`, il est correctement routé vers un modèle plus capable — et le « Pourquoi ? » l'explique en clair.

10. Carte des fichiers

Fichier	Rôle
extension/src/features.ts	Fonctions pures : <code>token_est</code> , langue, <code>has_code</code> , drapeaux, normalisation.
extension/src/signals.ts	<code>has_math</code> , drapeaux étendus, <code>normalizeModelLabel</code> , assemblage des signaux du prompt.
extension/src/conversationMemory.ts	Mémoire par fil : compteurs, <code>seen_*</code> , modèle courant ; réduction sans texte.
extension/src/mockRules.ts	Les 4 règles de décision (mode mock) + seuil de contexte long.
api/app/router.py	Routeur API stub (<code>HeuristicRouterV0</code> , 3 règles) + chiffrage coût/énergie.
extension/src/content-main.ts	Orchestration : signaux → reco → panneau → bascule ; <code>assist_mode</code> , acceptation différée, gardes.
extension/src/panel.ts / panelStyle.ts	Rendu du panneau et du badge (Shadow DOM, charte graphique, thèmes clair/sombre).
extension/src/modelSwitcher.ts	Bascule encadrée du modèle dans le sélecteur natif de claude.ai (vérifiée, sérialisée).
extension/CONVERGENCE_LEDGER.md	Journal de la finition multi-agents (13 rondes de jugement, ronde par ronde).

Note d'honnêteté : ces règles sont des **heuristiques v0**, volontairement simples et prudentes. Elles sont destinées à évoluer vers un routeur calibré côté serveur (Lot B), sans jamais renoncer aux deux invariants : *aucun texte ne quitte le poste* et *toute recommandation est explicable*.